

AppSec

Bootstrapping your first AppSec program.

Who am I?



Senior Product Security Swinger

 @honeybadgerhack

- Husband and Father
- Surfer
- Hydrofoiler
- Brazilian Jiu Jitsu Nerd
- Traveler
- Builder of things
- Destroyer of Code

Background:

- Ship Captain
- Pilot
- Owner of Fabrication and Engineering company
- Missionary
- Late change to tech

Tech:

- 1Password
- We hack Purple
- Bit Discovery
- Red Canary



Overview

We are going to outline building your AppSec program. With a different perspective on management and integration.

- Social Engineering for good!
- AppSec is internal sales.
- Be apart of the team.
- Technical SDLC dive.
- what has and hasn't worked for me

Warning!

These concepts and thoughts are based on anecdotal experiences from building application security programs. They are my personal opinions.

If you disagree that is perfectly fine.

If you need to email me to complain about my talk:

Youneedabetteruseofyourtimethantocomplainaboutmytalk@hotmail.com

If you find me abrasive and sarcastic but somewhat charming.....You are not wrong.

If you hang toilet paper over the back of the roll, please leave this talk now.



Syllabus

- What is an AppSec program?
- Why spend money here?
- How to Hire for these roles?
- Relationship with the company
- Threat Modeling
- Secure-SDLC
- Tools
- Metrics
- Education and training
- Continuous Improvement

What is AppSec?

Application Security or Product Security is a team / program designed to protect all the code you own.

That includes all the code your engineers write as well as the third party libraries or open source tools you use.

It's a strong relationship with engineering and C level stakeholders.

It's internal sales.

It's a smart use of tools and testing.

It's an understanding that if business doesn't flow there is no product to secure.

It's not:

Old school grumpy gate keeping.

Dumping 1k plus vulns on engineering and saying "Fix it".

Yelling when something can't get fixed right away.

Building a wall and starting a war with engineering.



Why Spend Money here?

- In 2020, the average cost to businesses affected by a data breach in the United States amounted to 8.64 million U.S. dollars.
- Reputational damage can be far worse than just the original dollar lost.
- You want to protect your customers.
- It is estimated that we lose 250 - 600 billion dollars intellectual property being stolen each year.
- Hopefully your insurance mandates it.
- Because it's not 2004 and you are a damn adult!



Hiring!

Hot controversial Take: Soft skills are more important than being a SME.



- If you hire a jerk nothing will get done.
- Your first hire should be and 50/50 mix of amazing soft skills and Application Security.
- High level understanding of AppSec is ok.
- While it is nice they don't need 10 years of dev experience.
- Stop trying to find that Unicorn for under industry standard pay.
- Balance your team with people who have experience in particular skill sets but still are jack of all trades.
- Good bridge building can make up for less technical skill but amazing technical skill can not make up for a bridge burner.
- AppSec has insane salaries. If you can't afford that take a chance on a passionate new person.
- Funny sarcastic people can be great bridge builders. We work primarily with engineers and it is a different audience than traditional infosec.

Hiring!

Opposing view

Hire rockstar security developers to fix all the bad code.

While this seems logical. Here are some problems I have noticed.

- These people are very hard to find and hire.
- There are not enough of them available so it doesn't scale well.
- The ones that do have good soft skills and teach usually start their own training business.
- They can easily become a block for engineering.
- Instead of a team mentality sometimes I see the "who is smartest competition" between staff / principle engineers and security engineers.

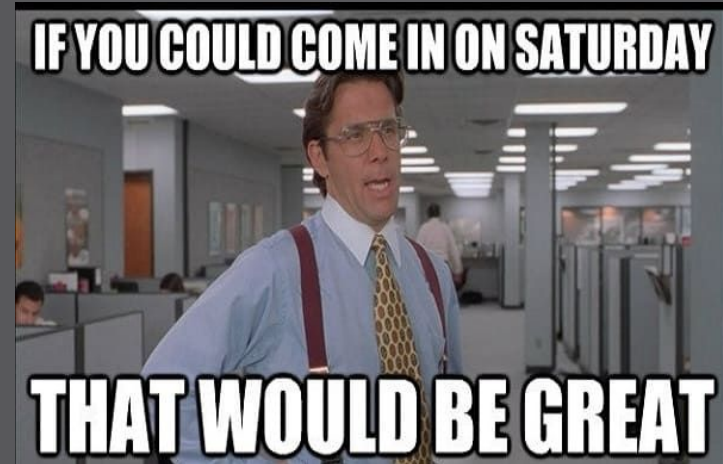


Leading Your Team

This is where many teams fail. A good security person isn't necessarily a good team leader. If your team leader doesn't genuinely care about the individuals success as well as the teams, you get shitty middle management....

A few quick things:

- Servant minded leadership
 - Customer Focused (team)
 - Focused on team members success
 - Leading from the front, doing the work
- Humility and honesty makes up for not knowing everything and gives team members a chance to shine.
- Honest Visibility, be open where you are crushing and where you need help.
- taking responsibility for failures.
- Learn to care about other people.....It is hard!



What is our relationship within the company?

- Every company will be different but here is how I try and place us.
- We view our positions as internal sales / Customer service. Similar to a modern IT program.
- We have a unique place where we touch C level, product managers, engineers, managers and IT. We also have relationships with sales and GRC.
- I want my team to be loved within the company. Not the trolls under the bridge.
- We should add value and security to everything we touch.

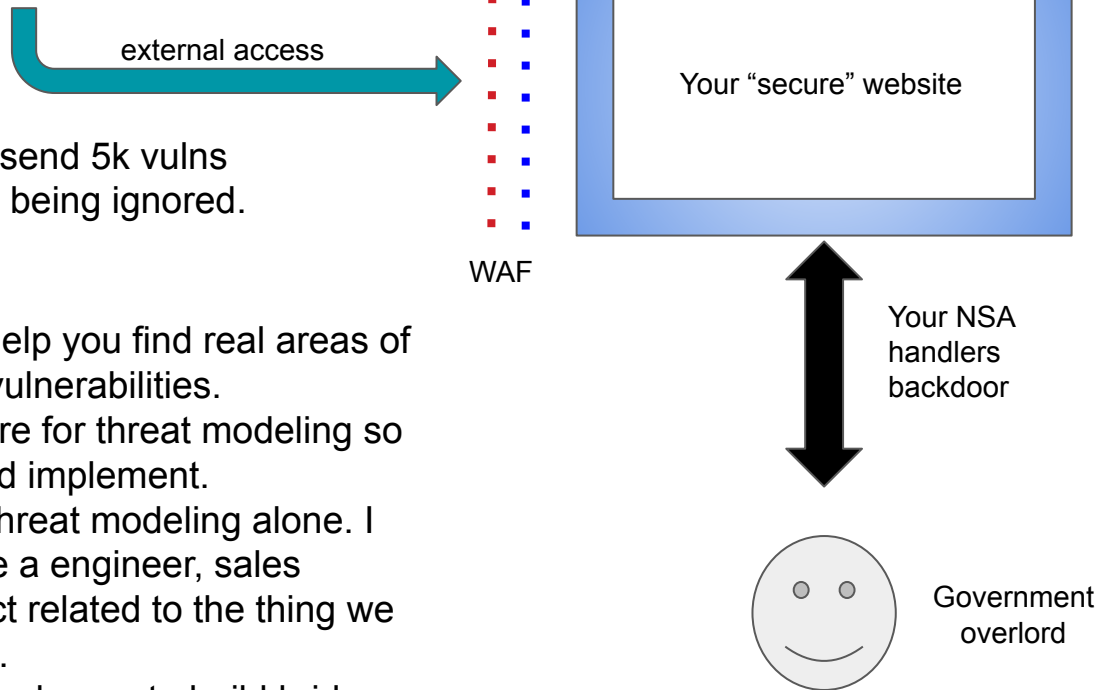


Where to Start? Threat Modeling.

Normal AppSec path is: get tool, run tool send 5k vulns to engineering and then bitch about them being ignored.
Don't Do This!

Threat modeling should help you find real areas of concern and exploitable vulnerabilities.

- Build out a procedure for threat modeling so others can learn and implement.
- You should not be threat modeling alone. I usually try and have a engineer, sales engineer or architect related to the thing we are threat modeling.
- This is an awesome chance to build bridges with engineering, especially if they have never done this before.
- Make it fun! Add beer, food and black hoodies.



Secure Software Development Life Cycle or Secure-SDLC

Secure-SDLC is implementing a set of controls to secure your software development process.

- Threat model first! You can't secure what you don't understand.
- Pushing left isn't always the right place to start.
- You need relationships built to implement these controls. If not you will get ignored.
- Start with one control. Plan out the next year and constantly adjust.
- You have to be flexible. Some things won't work in your work culture.
- After implementing a control step back and build a plan for continuous improvement for that control. Don't jump to the next one.
- If a control is a blocking control. Start it as a education and suggestion control. Slowly move it to blocking / mandatory. Make sure you have buy in and written policy to back it up.

Secure Software Development Life Cycle or Secure-SDLC

Steps for each control

- **Learn**
 - What problem are we going to solve. What technology will we use. What part of the company do we need to understand. Enumerate!
- **Plan**
 - Get all people involved and start to put together a good plan of action. Include implementation, Metrics, stakeholders, ect.
- **Design**
 - Look at how to implement this control in your company. A holistic approach.
- **Approval / Buy in**
 - You need stakeholders approval of your progress before executing.
- **Execute**
 - Triple check your plan. This is where people get pissed.
- **Improve**
 - This is where you fixed what pissed everyone off...or not.

	A	B	C	D	E	F	G
1	SDLC Stage	Security Control	Types of Technologies	Already owned?	Priority	Product Security Engineer	Notes
2	Architecture	Project Specific - Security Architectural Requirements	n/a		▼		Provide security requirements to architecture team, to enable devs. Encryption, authentication, etc
3		Security Architectural Standards	n/a		▼		Reference document for generally approved practices
4					▼		
5					▼		
6	Deliver / Plan	Feature/Enhancement Security Risk Assessments	n/a		▼		Review all new features and enhancements for their potential security impact
7		Threat Model	n/a		▼		Threat Modeling
8					▼		
9					▼		
10					▼		
11	Deliver / Develop	Engineering for pipeline security reviews	Brakeman, Fortify, Veracode, Checkmarx, ASOC		▼		Static Application Security Testing (SAST)
12			Black Duck, Snyk, White Source		▼		Software composition analysis (SCA)
13			Snyk, tfsec, Twistlock, etc		▼		Artifact/container scanning
14			KubeBench, DockerBench, Nessus CIS scans, some native AWS & Azure tools		▼		Configuration security reviews
15			n/a		▼		Pull request review (+1 or peer review)
16			Rubocop,		▼		Standardize and approve linters
17			Github Secrets, Container		▼		Secrets Detection
18					▼		
19					▼		
20	Deliver / Release	Run-time testing	Burp, Zap, Netsparker, etc, HCL ASOC		▼		Dynamic Application Security Testing (DAST)
21			Contrast, HCL ASOC		▼		Interactive Application Security Testing (IAST)
22			????		▼		Security functional / regression testing
23			n/a		▼		Manual testing (including penetration testing, validation of security requirements built in) by Red Canary engineers
24			n/a		▼		3rd party penetration testing
25			????		▼		Fuzzing
26			Nessus, Nexpose, Qualys, etc		▼		Network / vulnerability scanning
27					▼		
28					▼		
29	Deliver / Iterate	Production monitoring	Signal Sciences, Wallarm		▼		Web application firewall
30			CWP, Lacework, Threat Stack		▼		Endpoint protection
31			HackerOne, Bug Crowd		▼		Bug Bounty
32			Nessus, Nexpose, Qualys, etc. Threat Stack and AWS Inspector		▼		Network / vulnerability scanning
33			Splunk, CloudWatch, CloudTrail, GuardDuty, AWS Config		▼		Logging and monitoring
34					▼		
35		Vulnerability management			▼		Throughout the SDLC, vulnerabilities will be classified and managed to an SLA
36		Developer training / CTFs			▼		Train developers on a periodic and ongoing basis
37		Security Champions			▼		Implement security champions program
38		Rotating developer security education and security tech debt removal			▼		Create Security task force
39	Continuous	Secure coding standards			▼		Create document to outline our SCS and secure code snippets
40							
41							
42							
43							
44							

First Control!

This should be completely holistic and based on your company but here is some thoughts.

- Don't start with SAST (Static Application Security Testing). It has a lot of false positives and will piss off most engineers.
- Pick a control with LOW false positives!
- Writing policy to backup your control with buy in is a good idea. Secure Coding Standards.
- You need to be thoughtful. This sets the tone for your entire security program.
- You will piss someone off. Cookies and beer help.



Trigger Warning:

- Don't get dazzled by "SHIFT LEFT" concepts from sales guys.
- Sometimes you will be better off shifting right for the first control.
- Fewer false positives and exploitable vulnerabilities are found shifting right.
- Shifting left should still be a goal but sometimes it's easier to start right.



Next Controls

This is completely different for each team. Taking a holistic approach and doing a threat model style plan can be very helpful.

Do you have budget and a lot of code issues?

- You could try shifting left and look at SAST or SCA

Is it just you and you are overwhelmed?

- Security Champions or a Slackbot

Hired some folks?

- Embedded Security Engineers

Is there some fancy new product coming out?

- Threat Modeling or Security Risk Assessment



What worked for me?

A mix of DAST and Threat Modeling.

- Find an engineer you are buddies with and will be a Security Champion later.
- Grab a quick meeting and have a light Threat Model discussion.
- Take BURP Suite or Zap and run scan in an approved safe place based on threat model.
- Work very hard to filter out false positives.
- Have another longer meeting and go through the findings with that engineer to cancel false positives.
- Then bring the findings to engineering.



Embedded with engineering

When you have a more mature team and are growing, I have seen massive success with embedded controls. From a productivity, visual and social viewpoint these have been a massive success for us.

Embedded Security Engineer

- Security engineer who goes to some standups and all of planning meeting with an engineering team.
- Usually mapped 1 to 1 or 1 to 2.
- Has way more visibility into daily problems and code.
- Build personal relationships with engineers.
- Does threat models and SRAs with engineers they know.
- Becomes first point of contact for the team.
- Builds trust and visibility.

That awkward moment when you spawn in the enemy's base:



Tools!!!!

Tool acquisition is long and hard. Do it right.

- Don't trust the hype! Sales can be a sketchy.
- The less tools to do your job adequately the better. Looking at 17 different things a day is the path to dark side.
- Build a plan BEFORE you acquire tools. Having a centralized location for data and vulnerabilities will make achievable goals.
- Spend time testing tools before purchasing anything. Document the process and create a scoring system to be pragmatic about choosing.
- Test the tools API before buying. Some fancy tools have shit APIs.
- OWASP has a great chart with open source and commercial tools for all controls.
- When you get a tool. Implement, tune and improve before you get another tool. Take your time it is a slow process.

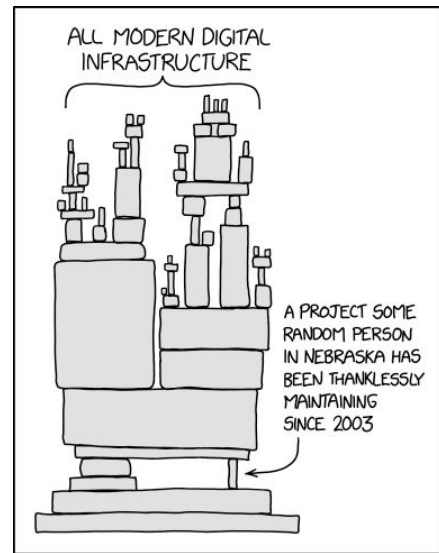
WHAT DO WE WANT!



No Budget No Toys!

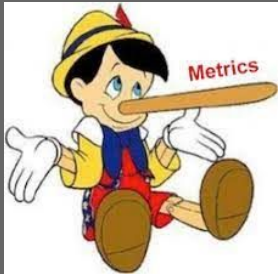
How do we get budget for an AppSec program?

- Sometimes you can't. It's you and the open source tools you implement.
- You can slowly show value.
 - Don't jam up engineering.
 - Plan everything out and implement controls slowly and efficiently
 - Keep metrics on your findings and remediation.
 - Build those bridges!
- Now with some value, learn to communicate to a business based audience. If you can show executives why financially it is good for the company, you will get closer to a real budget.
- Being able to communicate effectively with the C-suite and board what an exploit can cost the company is as important as talking to engineers.



USEFUL Metrics!

- Metrics are hard!
- Metrics should tell a story. Find out what stories you need to tell.
- Think about stories for your team's growth as well as benefiting executive, engineering ect.
- Find questions that will give you answers to your stories.
- Don't just throw a bunch of data on a slide.
- Do build metrics that will benefit continuous improvement.
- If done poorly metrics waste massive amounts of time and money.



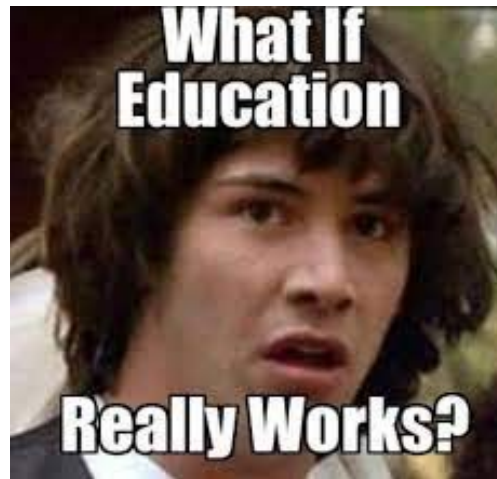
Education and Training

Education and Training might be the hardest but most important part of AppSec to do well.

Unfortunately you can't just tell all the engineers to learn AppSec. If you don't like security it is incredibly boring and a time suck.

So how do you implement an effective education program?

- You social engineer a security culture into your company.
- You build bridges with engineers and have real relationships.
- You find people passionate about teaching.
- You understand you are making a safer more profitable business.
- You are making more valuable engineers.



Social Engineering a Security Culture

Good Social Engineering

- Learns what is beneficial to both parties.
- Knows their audience.
- Has buy in from all parties needed.
- Builds a culture based on trust and visibility.
- Servant Leadership

Bad Social Engineering

- Self focused goals.
- Does not benefit all parties involved.
- Manipulation and zero visibility.
- Using the stick instead of the carrot.



Educational Programs

- New engineer onboarding apsec training.
- Lunch and learns based on new vulnerabilities or one found in your codebase.
- Building CTFs based on vulnerabilities pertaining to your codebase.
- Paid educational AppSec training programs.
- Specific training for security champions to help them grow into that roll.
- Non-AppSec security training that is fun and builds culture like lockpicking, physical penetration hacking, hardware hacking, ect.

Education, like most of security costs money or time. Look at the attributes you have on your team. Time, good repor, and public speaking? Try building out some fun lunch and learns. Unlikable but have budget? You can buy full programs to train engineers. I have found a mix of both can be very effective.

Continuous Improvement

One shitty control can ruin your whole program. You have to constantly improve everything you implement. Why would anyone take AppSec seriously if it doesn't seem like the team doesn't care? Mutual respect is kept by respecting engineering's time by making our tools run as perfect as possible.

- Try to refine the most obvious things first.
- Get a recurring meeting with an engineering manager to critique controls.
- Refine controls by studying your metrics.
- Automate Automate Automate!
- Be customer focused (engineering team)
- Be objective



Some ways forward.

You can't lay out a one size fits all plan to improve every security program but here are some things to try.

- Try to refine the most obvious things first.
- Get a recurring meeting with an engineering manager to critique controls.
- Refine controls by studying your metrics.
- Automate Automate Automate!
- Be customer focused (engineering team)
- Be objective
- Ask for a C suite review. They live and die by improvements and metrics.
- Build a team of honest people who are bought in. This will naturally lead to improvements.

Questions?

Warning: Answers may be opinion or flat out false. It is your fault for trusting a hacker.